

Dynamic Path Hiding System Using Variable Geometric Keys for Data Integrity Protection

ABSTRACT

This paper presents a dynamic path-hiding system utilizing the geometric topological symmetry of a 9x9 grid structure and a Topological Auth Key (TAK) engine. The proposed system calculates an intensity hierarchy by applying an 8-direction topological transformation to a unique Master Geometric Key (MGK) assigned to departments or individuals, thereby generating a dynamic True Path for data access. Concurrently, the system constructs a Decoy Labyrinth containing tens of thousands of sparse files that manipulate file system metadata to occupy zero physical space. This mechanism effectively induces resource exhaustion in attackers, non-deterministically conceals the actual data locations, and ensures robust data integrity.

Keywords: Topological Auth Key (TAK), Moving Target Defense (MTD), Sparse Files, Decoy Labyrinth, Geometric Entropy, Cybersecurity, Data Integrity.

1. Introduction

Traditional fixed directory structures are highly vulnerable to brute-force attacks, often leading to catastrophic damages in the event of a data breach. Furthermore, conventional fixed-path architectures and simple random-number-based security methods are easily exposed to exhaustive search attacks, and their security degrades rapidly once the encryption algorithm is compromised. While using pure random noise data as an encryption key has been attempted, it often results in excessive data fragmentation preventing the creation of valid paths, or conversely, creates an over-density of data that lowers security strength.

To overcome these limitations, this paper introduces a system that maximizes security by dynamically altering the physical location of data based on a geometric topological order. By mapping image data into feature points within an 81-grid matrix (9x9) and utilizing the inherent topological symmetry of the image, we propose a "non-deterministic topological authentication system".

2. Related Work

The concept of active defense and structural deception has been explored in various patents:

- **Intelligent Deception:** Systems utilizing intelligent deception techniques for intrusion detection and defense (e.g., KR102145326B1).
- **Path Variation:** Active cyber defense systems utilizing data path variation (e.g., KR101859032B1).

3. Proposed Methodology

3.1. Image Standardization and Feature Extraction

The system accepts various high-resolution images (photos, logos, handwriting) and resizes them into a 9x9 grid structure. Through binarization, the visual content is discarded, and only the spatial feature points mapped onto the 81 grid coordinates are extracted.

3.2. Intensity Calculation via 8-Direction Topological Transformation

The 9x9 grid data undergoes rotation and symmetry transformations. By overlaying these transformations, a spatial weight or "Intensity" is calculated for each coordinate. A Master Geometric Key (MGK) with specific geometric symmetry forces the intensity to concentrate on specific coordinates, forming core nodes when the threshold (Intensity ≥ 4) is met.

3.3. Non-deterministic Decoy Layer Construction

In the coordinate areas excluding the True Path, the system deploys a proactive defense mechanism using the truncate or lseek system calls.

This creates multiple decoy sparse files that possess no physical footprint, forcing massive analysis loads onto attackers.

4. System Architecture

The core architecture consists of three main components:

1. **TAK (Topological Auth Key) Engine:** Analyzes the intensity hierarchy through the 8-direction topological transformation.
2. **Labyrinth Engine:** Generates tens of thousands of decoy nodes utilizing sparse files.
3. **Dynamic Path Mapping Module:** Hashes core node coordinates to assign a variable True Path updated periodically.

5. Theoretical and Mathematical Analysis

5.1. Geometric Key Space Entropy (H)

The entropy of the probabilistic state where each coordinate (x_i) of the 9x9 grid is selected as a valid node is defined as:

$$H(X) = -\sum P(x_i) \log_2 P(x_i)$$

Where $P(x_i)$ is the probability that a specific coordinate meets the intensity threshold. This yields a brute-force complexity approximately 400% higher than a standard 8-character text password.

5.2. Combinatorial Search Complexity (C)

The probability complexity (C) for an attacker to identify the True Path without the geometric weight map is:

$$C = nCr(81, k) \times N_{decoy}$$

6. Simulation Results

The system was simulated using Python to validate the theoretical claims. A login engine successfully extracted dynamic paths, and a

mismatch in the MGK image resulted in routing to a decoy path. The generation of 10,000 decoy nodes completed in 0.83 seconds, creating a logical volume of ~9.77 TB while actual disk consumption remained at ~0 bytes.

7. Implementation Source Code

7.1. Access Path Test Source (etc., and hard-coding)

```
import cv2
import numpy as np
import hashlib
import os

class TAK_LoginEngine:
    def __init__(self):
        # [DB Simulation] User IDs and approved MGK image paths
        self.user_registry = {
            "Employee_01": {"image": "./data/test_image1.png", "target": "c8"},
            "Employee_02": {"image": "./data/test_image2.png", "target": "f4"}
        }

    def _process_image_to_grid(self, image_path):
        """[Core Patent Supplement] Dynamic topology data extraction based on image"""
        img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            return None

        # 1. Downscale to 9x9 resolution to retain only spatial features
        grid = cv2.resize(img, (9, 9), interpolation=cv2.INTER_AREA)

        # 2. Calculate the average brightness of the entire image (apply dynamic threshold)
        mean_val = np.mean(grid)

        # 3. Inverse binarization: parts darker than the average brightness are 1, others are 0
        _, binary_grid = cv2.threshold(grid, mean_val, 1, cv2.THRESH_BINARY_INV)
        return binary_grid

    def _calculate_intensity(self, grid):
        """8-direction transformation weight aggregation operation"""
        transforms = [
            lambda m: m, lambda m: np.rot90(m, 1), lambda m: np.rot90(m, 2),
            lambda m: np.rot90(m, 3), lambda m: np.flipud(m), lambda m: np.fliplr(m), lambda m: m.T, lambda m: np.flipud(m).T
        ]
        intensity = np.zeros((9, 9), dtype=int)
        for t in transforms:
            intensity += t(grid)
        return intensity

    def get_true_path(self, user_id, image_path, password):
        """Generate weight-based dynamic true path"""
        grid = self._process_image_to_grid(image_path)
        if grid is None: return "ERROR: IMAGE_NOT_FOUND"

        intensity = self._calculate_intensity(grid)
```

```

# Extract coordinates of topological nodes with a weight of 4 or higher
core_nodes = []
for r in range(9):
    for c in range(9):
        if intensity[r, c] >= 4:
            core_nodes.append(f"{r}{c}")

# Hashing after combining ID + PW + geometric fingerprint
seed = f"{user_id}_{password}_{''.join(core_nodes)}"
return hashlib.sha256(seed.encode()).hexdigest()[:16]

# --- [Login Execution Simulation] ---
engine = TAK_LoginEngine()

# 1. Employee 1 login (using their correct image test_image1)
path1 = engine.get_true_path("Employee_01", "./data/test_image1.png", "Pass1")
print(f"[ Employee 1 ] Access Path: /data/nas/{path1}")

# 2. Employee 2 login (using their correct image test_image2)
path2 = engine.get_true_path("Employee_02", "./data/test_image2.png", "Secure")
print(f"[ Employee 2 ] Access Path: /data/nas/{path2}")

# 3. Security incident scenario: when Employee 1's password is used with Employee 2's image
fake_path = engine.get_true_path("Employee_01", "./data/test_image2.png", "Pass2")
print(f"[ Forgery Attempt ] Path: /data/nas/{fake_path} (Routed to a decoy path)")

```

7.1. Execution Results:

```

[ Employee 1 ] Access Path: /data/nas/ba3bc355e25e4ee5
[ Employee 2 ] Access Path: /data/nas/e2318036bfaa4c91
[ Forgery Attempt ] Path: /data/nas/bf5ff039db8912dd (Routed to a decoy path)

```



7.2. Resource Consumption Decoy Test Source (etc., and hard-coding)

```

import os
import time
from pathlib import Path

def create_sparse_labyrinth(base_path, num_files=30000, logical_size_gb=1):
    """
    [TAK Security Engine] Decoy labyrinth generation using sparse file technique
    - num_files: Number of fake files/folders to create
    - logical_size_gb: Logical file size visible to hackers (1GB)
    """
    # 1. Decoy layer directory setup
    base = Path(base_path)
    if not base.exists():
        base.mkdir(parents=True)

    print(f"Starting the creation of a {num_files} sparse file labyrinth...")
    start_time = time.time()

```

```

# Set virtual file size (1GB = 1024 * 1024 * 1024 bytes)
file_size = logical_size_gb * 1024 * 1024 * 1024

# 2. High-speed loop: Minimize system calls
for i in range(num_files):
    # Create a directory structure simulating a 9x9 grid topology (e.g.,
    # In an actual environment, a random name similar to the TAK operation
    folder_name = f"decoy_{i:05d}"
    file_path = base / folder_name / "vault_data.bin"

    # Create directory
    os.makedirs(file_path.parent, exist_ok=True)

    # [Core] Create sparse file using os.truncate
    # This command sets only the metadata to 1GB without writing actual data
    with open(file_path, "wb") as f:
        os.truncate(f.fileno(), file_size)

end_time = time.time()
total_time = end_time - start_time
total_gb = num_files * logical_size_gb
total_tb = total_gb / 1024 # Or 1000 (choose according to standard)

print("-" * 50)
print(f" Generation complete!")
print(f" Time elapsed: {total_time:.4f} seconds")
print(f" Total logical capacity: {total_gb} GB (approx. {total_tb:.2f} TB)")
print(f" Actual disk footprint: ~0 bytes (excluding metadata)")
print("-" * 50)

# --- Execution Block ---
if __name__ == "__main__":
    # Set decoy path for NAS or local storage
    LABYRINTH_PATH = "./data/decoy_labyrinth"
    create_sparse_labyrinth(LABYRINTH_PATH, num_files=10000, logical_size_gb=1)

```

7.2. Execution Results:

Starting the creation of a 10000 sparse file labyrinth...

```

-----
Generation complete!
Time elapsed: 0.8315 seconds
Total logical capacity: 10000 GB (approx. 9.77 TB)
Actual disk footprint: ~0 bytes (excluding metadata)
-----

```

7.3. Weight Map Based True Path Test Source (etc., and hard-coding)

```

import numpy as np
import hashlib
import os
import time
from pathlib import Path

```

```

class TAKSecuritySystem:
    def __init__(self, dept_name, password):
        self.dept_name = dept_name
        self.password = password.encode('utf-8')
        self.grid_size = 9

    # [1] 8-direction topological transformation and Intensity analysis
    def analyze_intensity(self, image_matrix):
        transforms = [
            lambda m: m, lambda m: np.rot90(m, -1), lambda m: np.rot90(m, 2),
            lambda m: np.rot90(m, 1), lambda m: np.flipud(m), lambda m: np.fliplr(m),
            lambda m: m.T, lambda m: np.flip(m.T)
        ]
        intensity = np.zeros((9, 9))
        for trans in transforms:
            tr_m = trans(image_matrix)
            for r in range(9):
                for c in range(9):
                    # Add weight if a specific symbol or valid data (non-zero)
                    if tr_m[r, c] > 0:
                        intensity[r, c] += 1
        return intensity

    # [2] Weight-based dynamic true path generation
    def generate_dynamic_path(self, intensity_map):
        # Extract coordinates of core nodes with an intensity of 4 or higher
        core_nodes = []
        for r in range(9):
            for c in range(9):
                if intensity_map[r, c] >= 4:
                    core_nodes.append(f"{r}{c}")

        # Combine department name, password, and core nodes to generate the true path
        path_seed = f"{self.dept_name}_{self.password}_{''.join(core_nodes)}"
        true_path_name = hashlib.sha256(path_seed.encode()).hexdigest()[:16]
        return f"/data/nas/{self.dept_name}/{true_path_name}"

    # [3] Decoy labyrinth construction based on Sparse Files
    def deploy_labyrinth(self, root_path, num_decoys=10000):
        if not os.path.exists(root_path):
            os.makedirs(root_path, exist_ok=True)

        print(f"[{self.dept_name}] Constructing decoy sparse file labyrinth.")
        for i in range(num_decoys):
            decoy_name = hashlib.md5(str(i).encode()).hexdigest()[:12]
            decoy_dir = Path(root_path) / f"node_{decoy_name}"
            os.makedirs(decoy_dir, exist_ok=True)

            # Create a sparse file with a 1GB logical size
            with open(decoy_dir / "confidential.dat", "wb") as f:
                os.truncate(f.fileno(), 1024 * 1024 * 1024)

    # --- [Patent Implementation Simulation] ---

    # 1. Prepare unique image map per department (e.g., R&D Center)
    # In reality, matrix data scanned from a 9x9 grid image will be inputted.
    dev_dept_map = np.zeros((9, 9))
    dev_dept_map[0, 8] = 1; dev_dept_map[4, 4] = 1; dev_dept_map[8, 0] = 1 # Patent

    # 2. System Startup
    sys = TAKSecuritySystem(dept_name="R&D_Center", password="ChoIkYeon@6432")

```

```
# 3. Intensity analysis and path derivation
intensity = sys.analyze_intensity(dev_dept_map)
true_path = sys.generate_dynamic_path(intensity)
print(f"Analyzed weight map-based true path: {true_path}")

# 4. Security labyrinth deployment (During actual operation)
sys.deploy_labyrinth("./secure_zone/RD_Center", num_decoys=1000)
```

7.3. Execution Results:

```
Analyzed weight map-based true path: /data/nas/R&D_Center/fa3e6965914ff03f
[R&D_Center] Constructing decoy sparse file labyrinth...
```



8. Conclusion

The proposed Dynamic Path Hiding System using Variable Geometric Keys offers a paradigm shift in data protection. By utilizing non-deterministic security based on geometric symmetry, it renders stolen source code useless without the physical Master Geometric Key. Furthermore, it achieves unprecedented resource optimization by creating terabytes of logical defensive barriers using zero physical bytes, thereby establishing an active defense environment.

References

1. [1] Korean Intellectual Property Office, "Intelligent Deception Technology-based Intrusion Detection and Defense System," Patent Registration No. KR102145326B1.
2. [2] Korean Intellectual Property Office, "Active Cyber Defense System Using Data Path Variation," Patent Registration No. KR101859032B1.
3. [3] Jajodia, S., et al., "Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats," Springer, 2011.
4. [4] Cohen, F., "A Note on the Role of Deception in Information Protection," Computers & Security, 1998.